SIMATS ENGINEERING

SIMATS Online Programming Lab

DS PROGRAMMING

N Thejasree
192524438RC

QUESTIONS 10 / 10 completed

Queue Using Array

QUEUE USING ARRAYS

Write a C program to implement a Queue using Arrays with operations such as insertion (enqueue), deletion (dequeue), display, and peek. The program should simulate a ticket booking counter where customers arrive in sequence and are served on a first-come-first-served basis. The queue should maintain the order of customer requests and display the current

PROGRAM EDITOR

C

Run Save

```c
#include <stdio.h>

int main()
{
    int q[10];
    int front = 0, rear = -1;

    q[++rear] = 101;
    q[++rear] = 102;
    q[++rear] = 103;

    printf("Queue Elements: ");
    for (int i = front; i <= rear; i++)
        printf("%d ", q[i]);

    printf("\nDeleted Element: %d", q[front++]);

    printf("\nQueue Elements: ");
    for (int i = front; i <= rear; i++)
        printf("%d ", q[i]);

}
```

OUTPUT

```
Queue Elements: 101 102 103
Deleted Element: 101
Queue Elements: 102 103
```

INPUT

Your INPUT goes here!

SIMATS ENGINEERING

SIMATS Online Programming Lab

DS PROGRAMMING

N Thejasree
192524438IC

QUESTIONS 10 / 10 completed

Queue Using Linke

QUEUE USING LINKED LISTS

Write a C program to implement a Queue using Linked Lists for managing patients waiting in a hospital consultation room. The program should support enqueue, dequeue, display, and front operations without any fixed size limitation. Sample Input: Insert Patient IDs 201, 202, 203, Display, Delete one patient. Sample Output: Queue Elements: 201 202 203. Deleted Patient ID: 201. Remaining Queue: 202 203.

PROGRAM EDITOR

C

Run Save

```c
#include <stdio.h>
#include <stdlib.h>

struct Node
{
    int data;
    struct Node *next;
};

int main()
{
    struct Node *front = NULL;
    struct Node *rear = NULL;
    struct Node *p;

    int a[] = {201, 202, 203};
    int i;

    for (i = 0; i < 3; i++)
    {
        p = (struct Node *)malloc(sizeof(struct Node));
```

OUTPUT

```
Queue Elements: 201 202 203
Deleted Patient ID: 201
Remaining Queue: 202 203
```

INPUT

Your INPUT goes here!

SIMATS ENGINEERING

SIMATS Online Programming Lab

DS PROGRAMMING

N Thejasree
192524438RC

QUESTIONS 10 / 10 completed

Double Ended Que

DOUBLE ENDED QUEUE

Write a C program to implement a Double Ended Queue (Deque) using Arrays where insertion and deletion can be performed from both front and rear ends. Consider a warehouse inventory management system where goods can be added or removed from either side based on priority. Sample Input: Insert Front 100, Insert Rear 200, Insert Front 50, Delete Rear. Sample Output: Deleted Element: 200. Queue Elements: 50 100.

PROGRAM EDITOR

C

Run Save

```c
#include <stdio.h>

int main()
{
    int q[10];
    int front = 4, rear = 3;
    int deleted;

    front--;
    q[front] = 100;

    rear++;
    q[rear] = 200;

    front--;
    q[front] = 50;

    deleted = q[rear];
    rear--;

    printf("Deleted Element: %d\n", deleted);
```

OUTPUT

```
Deleted Element: 200
Queue Elements: 50 100
```

INPUT

Your INPUT goes here!

SIMATS ENGINEERING

SIMATS Online Programming Lab

DS PROGRAMMING

N Thejasree
192524438RC

QUESTIONS 10 / 10 completed

Priority Queue Usi

PRIORITY QUEUE USING ARRAYS

Write a C program to implement a Priority Queue using Arrays for processing emergency service requests. Requests with higher priority must be processed before lower-priority requests irrespective of arrival order. Sample Input: Insert (101, Priority 3). Insert (102, Priority 1). Insert (103, Priority 2). Delete. Sample Output: Processed Request: 102. Remaining Queue: 103 101.

PROGRAM EDITOR

C

Run Save

```c
#include <stdio.h>

int main()
{
    int item[3] = {101, 102, 103};
    int priority[3] = {3, 1, 2};
    int i, j, temp;

    for (i = 0; i < 2; i++)
    {
        for (j = i + 1; j < 3; j++)
        {
            if (priority[i] > priority[j])
            {
                temp = priority[i];
                priority[i] = priority[j];
                priority[j] = temp;

                temp = item[i];
                item[i] = item[j];
                item[j] = temp;
```

OUTPUT

```
Processed Request: 102
Remaining Queue: 103 101
```

INPUT

Your INPUT goes here!

SIMATS ENGINEERING

SIMATS Online Programming Lab

DS PROGRAMMING

N Thejasree
192524438RC

QUESTIONS 10 / 10 completed

Queue Data Struct

QUEUE DATA STRUCTURE

Write a C program to simulate a Railway Reservation Waiting List using Queue data structure. Passengers enter the waiting queue and are allocated seats as cancellations occur. The program should maintain passenger details and process requests in FIFO order. Sample Input: Add Passenger 501, Add Passenger 502, Cancellation Occurs, Display Queue. Sample Output: Passenger 501 Confirmed. Waiting List: 502.

PROGRAM EDITOR

C

Run Save

```c
#include <stdio.h>

int main()
{
    int queue[2];
    int front = 0, rear = 1;

    queue[0] = 501;
    queue[1] = 502;

    printf("Passenger %d Confirmed.\n", queue[front]);

    front = front + 1;

    printf("Waiting List: %d", queue[front]);

    return 0;
}
```

OUTPUT

```
Passenger 501 Confirmed.
Waiting List: 502
```

INPUT

Your INPUT goes here!

SIMATS ENGINEERING

SIMATS Online Programming Lab

DS PROGRAMMING

N Thejasree
192524438RC

QUESTIONS 10 / 10 completed

Queue

QUEUE

Write a C program to implement a Queue for managing online food delivery orders where orders are received and processed in the order they arrive. The program should allow adding new orders, processing completed orders, and displaying pending orders. Sample Input: Add Orders 1001, 1002, 1003, Process Order. Sample Output: Processed Order: 1001. Pending Orders: 1002 1003.

PROGRAM EDITOR

C

Run Save

```c
#include <stdio.h>

#define MAX 100

int main()
{
    int queue[MAX];
    int front = 0;
    int rear = -1;


    queue[++rear] = 1001;
    queue[++rear] = 1002;
    queue[++rear] = 1003;


    if (front <= rear)
    {
        printf("Processed Order: %d\n", queue[front]);
        front++;
    }
```

OUTPUT

```
Processed Order: 1001
Pending Orders: 1002 1003
```

INPUT

Your INPUT goes here!

SIMATS ENGINEERING

SIMATS Online Programming Lab

DS PROGRAMMING

N Thejasree
192524438RC

QUESTIONS 10 / 10 completed

Queue Using Linke

QUEUE USING LINKED LISTS -2

Write a C program to implement a Queue using Linked Lists for managing network packets in a router. Packets should be transmitted in the order received, and the program must support dynamic packet insertion and removal. Sample Input: Insert Packets 11, 12, 13, Transmit Packet. Sample Output: Transmitted Packet: 11. Remaining Packets: 12 13.

PROGRAM EDITOR

C

Run Save

```c
#include <stdio.h>
#include <stdlib.h>

struct Node
{
    int data;
    struct Node *next;
};

int main()
{
    struct Node *front = NULL;
    struct Node *rear = NULL;
    struct Node *newNode;
    struct Node *temp;

    /* Insert Packets 11, 12, 13 */
    newNode = (struct Node *)malloc(sizeof(struct Node));
    newNode->data = 11;
    newNode->next = NULL;
    front = rear = newNode;
```

OUTPUT

```
Transmitted Packet: 11
Remaining Packets: 12 13
```

INPUT

Your INPUT goes here!

SIMATS ENGINEERING

SIMATS Online Programming Lab

DS PROGRAMMING

N Thejasree
192524388RC

QUESTIONS 10 / 10 completed

Circular Queue

CIRCULAR QUEUE

Write a C program to implement a Circular Queue for a traffic signal management system where vehicles are processed lane by lane in a cyclic manner. The system should continuously rotate through available lanes while maintaining queue order. Sample Input: Vehicles 21, 22, 23 arrive, one vehicle passes. Sample Output: Vehicle Passed: 21. Queue: 22 23.

PROGRAM EDITOR

C

Run Save

```c
#include <stdio.h>

#define MAX 100

int queue[MAX];
int front = -1, rear = -1;

// Function to add a vehicle to the circular queue
void enqueue(int value) {
    if ((rear + 1) % MAX == front) {
        return; // Queue is full
    }
    if (front == -1) {
        front = 0;
    }
    rear = (rear + 1) % MAX;
    queue[rear] = value;
}

// Function to remove a vehicle from the circular queue
int dequeue() {
```

OUTPUT

INPUT

Your INPUT goes here!

SIMATS ENGINEERING

SIMATS Online Programming Lab

DS PROGRAMMING

N Thejasree
192524438RC

QUESTIONS 10 / 10 completed

Queue-2

QUEUE-2

Write a C program to implement a Queue for managing examination answer sheet evaluation. Answer sheets submitted by students are evaluated in the same sequence in which they are received. Sample Input: Sheet IDs 401, 402, 403 submitted, evaluate one sheet. Sample Output: Evaluated Sheet: 401. Pending Sheets: 402 403.

PROGRAM EDITOR

C

Run Save

```c
#include <stdio.h>

int main() {
    int q[100];
    int f = 0, r = 0;
    int s1, s2, s3;

    if (scanf("%d %d %d", &s1, &s2, &s3) == 3) {
        q[r++] = s1;
        q[r++] = s2;
        q[r++] = s3;

        int eval = q[f++];

        printf("Evaluated Sheet: %d. Pending Sheets:", eval);
        for (int i = f; i < r; i++) {
            printf(" %d", q[i]);
        }
    }
    return 0;
}
```

INPUT

```
401 402 403
```

OUTPUT

```
Evaluated Sheet: 401. Pending Sheets: 402 403
```

SIMATS ENGINEERING

SIMATS Online Programming Lab

DS PROGRAMMING

N Thejasree
192524438RC

QUESTIONS 10 / 10 completed

Queue Using Linke

QUEUE USING LINKED LIST-3

Write a C program to simulate a supermarket billing queue using Linked List implementation. Customers join the billing queue and are billed in FIFO order. The queue size should dynamically increase as customers arrive. Sample Input: Customers 11, 12, 13 enter, one customer billed. Sample Output: Billed Customer: 11. Remaining Customers: 12 13.

PROGRAM EDITOR

C

Run Save

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

struct Node *front = NULL, *rear = NULL;

void enqueue(int val) {
    struct Node* temp = (struct Node*)malloc(sizeof(struct Node));
    temp->data = val;
    temp->next = NULL;
    if (rear == NULL) {
        front = rear = temp;
        return;
    }
    rear->next = temp;
    rear = temp;
}
```

OUTPUT

```
Billed Customer: 11. Remaining Customers: 12 13
```

INPUT

```
11 12 13
```